

Edufyi Tech Solutions

Website — Technical & Architecture Documentation

Full-stack Next.js application: frontend, backend, authentication & payment flow, database schema, dependencies, and storage requirements.

Prepared for	Edufyi Tech Solutions — edufyitechsolutions.in
Framework	Next.js 16.3.1 (App Router, Turbopack)
Language	TypeScript 5
Database	Neon Serverless PostgreSQL
Hosting model	Node.js-capable host (Vercel / Netlify / similar)

1. Project Overview

Edufyi Tech Solutions is a marketing + e-learning enrolment website built entirely on **Next.js 16** (App Router) using **one single codebase for both frontend and backend** — there is no separate backend server/repo. Pages are React Server Components rendered on the server (SSR/ISR); backend logic lives in Next.js **Route Handlers** under `src/app/api/*`, running on the Node.js runtime so they can use real server-only packages (database driver, crypto, email, payment SDK).

The site showcases programs/courses, captures leads (Register Interest, Contact, B2B), sells paid plans via Razorpay, and gives paying students a session-authenticated `/dashboard`. Content (programs, testimonials, career paths, success stories) is read from a Postgres database when configured, and automatically falls back to static seed data (`src/lib/seed.ts`) when it isn't — so the site always renders even before a database is connected.

2. Technology Stack

2.1 Frontend

Layer	Technology	Purpose
Framework	Next.js 16.3.1 (App Router, Turbopack)	Routing, SSR, ISR, bundling, dev server
UI library	React 19.2.8 / React DOM 19.2.8	Component rendering
Language	TypeScript 5	Static typing across app + API
Styling	Tailwind CSS 4 (+ <code>@tailwindcss/postcss</code>)	Utility-first CSS, design tokens
Animation	motion 13 (Framer Motion)	Scroll-story, transitions, reveals
Smooth scroll	lenis 1.3	Inertia/smooth page scrolling
Icons	lucide-react	Icon set used across UI
Fonts/Assets	<code>next/font</code> , <code>next/image</code> , <code>public/*</code>	Optimised fonts + images

2.2 Backend (inside the same Next.js app)

Layer	Technology	Purpose
Runtime	Node.js (Next.js Route Handlers)	<code>src/app/api/*</code> — server-only logic
Database driver	<code>@neondatabase/serverless 1.1</code>	HTTP/WebSocket driver for Neon Postgres
Auth	Node crypto (scrypt + HMAC-SHA256)	Password hashing + signed session cookies — no 3rd-party auth library
Payments	razorpay 2.9 SDK	Order creation + payment verification
Email	nodemailer 9	SMTP transactional + admin-alert emails
Access gate	<code>src/proxy.ts</code> (Next.js middleware)	Protects <code>/dashboard/*</code> using the session cookie
Rate limiting	<code>src/lib/rateLimit.ts</code> (custom, in-memory)	Throttles login/contact/payment endpoints

2.3 Database & Storage

Concern	Solution
Structured data	Neon Serverless PostgreSQL (managed, cloud, pay-as-you-go)
Media / static assets	Files under public/ (images, videos, PDFs) — served by the host, not the database
File uploads by users	None implemented — no user-generated file storage exists today
Sessions	Stateless — no session table; the signed cookie itself is the source of truth

2.4 Tooling / Dev dependencies

Package	Purpose
typescript, @types/*	Type-checking for TS/React/Node APIs
eslint, eslint-config-next	Linting (npm run lint)
tailwindcss, @tailwindcss/postcss	CSS build pipeline

3. node_modules — Dependency Footprint

node_modules is never committed to git (excluded via .gitignore) — it is regenerated on any machine/host by running `npm install`, which reads `package.json` + the locked versions in `package-lock.json`.

Metric	Value
Installed size on disk (local dev)	≈ 455 MB
File count	≈ 26,700 files
Direct production dependencies	8 packages (see 2.1 / 2.2 tables)
Direct dev dependencies	9 packages
Install command	npm install
Build command	npm run build (next build)
Run command (production)	npm run start (next start)
Dev command	npm run dev (next dev --turbo)

4. Project Folder Structure

```

edufy-tech/
├── src/
│   ├── app/
│   │   ├── page.tsx           # Next.js App Router (pages + API)
│   │   ├── programs/[slug]/   Home
│   │   ├── pricing/           Program detail pages
│   │   ├── register/          Plans + Razorpay checkout UI
│   │   ├── login/             "Register Interest" lead form
│   │   ├── dashboard/         Login form
│   │   ├── career-paths/[domain]/ Authenticated student dashboard (proxy-gated)
│   │   ├── partners/, about/, contact/, success-stories/, legal/, employee/, aira/ Career path detail pages
│   │   └── api/                # Backend route handlers (Node runtime)
│   │       ├── auth/login, auth/logout, auth/me
│   │       ├── payment/create-order, payment/verify
│   │       └── register-interest, contact, b2b
│   ├── components/            # React UI components (Navbar, Footer, forms, ScrollStory, ...)
│   ├── ui/                    # Small shared primitives (buttons etc.)
│   └── lib/                    # Server + shared logic (no UI)
│       ├── db.ts              Neon queries + seed fallback
│       ├── auth.ts             Password hashing + session tokens
│       ├── razorpay.ts         Order creation + signature verification
│       ├── mail.ts             SMTP transactional emails
│       └── rateLimit.ts        In-memory request throttling

```


5. System Architecture



Everything above runs as **one deployable Next.js app** — there is no separate backend service to host. External systems are: Neon (database), Razorpay (payments), and an SMTP provider (email) — all reached only from server-side code, never exposed to the browser.

6. Authentication, Sessions & Payment Flow

Important: the codebase currently has **one account type only** — regular users in the users table. There is no separate "admin" login, role column, or admin panel. ADMIN_EMAIL / PARTNERSHIPS_EMAIL are just *destination inboxes* for lead-notification emails (Contact / B2B / Register-Interest submissions) — they are not accounts with elevated access. If a staff-only admin dashboard is needed, it does not exist yet and would need to be built (e.g. add a role column + a protected /admin route).

6.1 Flow A — Existing user logging in

- 1 User submits email + password on /login.
- 2 Browser calls POST /api/auth/login (rate-limited: 8 requests/min per IP).
- 3 Server looks up the user by email in the users table.
- 4 Password is checked with **scrypt** (memory-hard hash) using a timing-safe comparison — never stored or compared in plain text.
- 5 On success, the server creates a **signed session token**: payload {userId, email, fullName, exp} base64url-encoded, signed with HMAC-SHA256 using the server-only AUTH_SECRET.
- 6 Token is set as an edufy_session cookie: HttpOnly, Secure in production, SameSite=Lax, 30-day expiry.

- 7 Any later request to `/dashboard/*` is intercepted by `src/proxy.ts`, which re-verifies the cookie's signature + expiry before allowing the page to render; invalid/missing → redirected to `/login`.

6.2 Flow B — New student created automatically via payment

There is no separate "Sign Up" form — paying for a plan is what creates an account.

- 1 Visitor picks a plan on `/pricing` → `POST /api/payment/create-order`. The price is resolved **server-side** from `src/lib/pricing.ts` — the client only sends a plan ID, never an amount, so price tampering isn't possible.
- 2 Razorpay Checkout opens in the browser using the returned `orderId`.
- 3 After payment, the client posts `razorpay_order_id / payment_id / signature` to `POST /api/payment/verify`.
- 4 Server recomputes the HMAC-SHA256 signature from `orderId|paymentId` using `RAZORPAY_KEY_SECRET` and compares it — rejects any tampered request.
- 5 Server re-fetches the order from Razorpay's API to read back the **trusted** plan/course notes (set when the order was created) — the client cannot claim it bought something else.
- 6 Server finds the user by email, or **auto-creates one** with a randomly generated password (hashed with `scrypt` before storing — never emailed or logged in plain text form to storage, only sent once to the buyer's inbox).
- 7 An `enrollments` row is inserted, linking user → plan/course/amount/payment IDs.
- 8 The same signed session cookie is issued — the student lands logged-in on `/dashboard` immediately.
- 9 A best-effort confirmation email is sent via SMTP (includes the generated password for brand-new accounts).

7. Database Schema (Neon PostgreSQL)

Applied via `npm run db:migrate`, which runs `scripts/schema.sql` against `DATABASE_URL`. All statements are idempotent (`CREATE TABLE IF NOT EXISTS`).

courses — program/course catalogue (falls back to seed.ts if empty)

Column	Type	Notes
slug	text PK	URL identifier, e.g. ai-ml
title, category, level, duration	text	Display fields
price_inr	integer	Listed price
rating	numeric(2,1)	Default 4.5
short_desc, description	text	Copy
syllabus, tags, projects, mentors, tools, outcomes	jsonb	Structured lists, default []
image_url	text	Cover image path
is_featured	boolean	Homepage featured flag
created_at	timestampz	Default now()

testimonials / career_paths / success_stories

Column	Type	Notes
testimonials	id, name, role, company, quote, avatar_url, rating	Homepage/testimonials carousel
career_paths	id, domain (unique), tagline, roles, industries, progression (jsonb)	Career-path pages
success_stories	id, name, program, domain, outcome, quote, linkedin_url, company, avatar_url	Success-stories page

Lead-capture tables — written by /api/* route handlers

Column	Type	Notes
register_interest	program_slug, full_name, email, phone, discount_code, message	From /register
contact_leads	full_name, email, phone, subject, message	From /contact
b2b_leads	org_name, org_type, contact_name, email, phone, interest, message	From B2B partnership form

users — real accounts (login/registration backend)

Column	Type	Notes
id	uuid PK, gen_random_uuid()	—
full_name, email (unique), phone	text	email indexed lower(email)
password_hash	text	":" — never plain text
created_at	timestampz	Default now()

enrollments — one row per purchased plan (post-payment)

Column	Type	Notes
id	uuid PK	—
user_id	uuid, FK → users(id) ON DELETE CASCADE	Indexed
course_slug, course_title, plan_id, plan_name	text	What was bought
amount_inr	integer	Amount actually charged
order_id, payment_id	text	Razorpay identifiers
created_at	timestampz	Default now()

8. Backend API Routes

Method & Path	Purpose	Protection
POST /api/auth/login	Verify credentials, issue session cookie	Rate-limited (8/min/IP)
POST /api/auth/logout	Clear the session cookie	None needed
GET /api/auth/me	Return current session's user (or null)	Reads cookie only
POST /api/register-interest	Save a lead + notify admin email	Rate-limited
POST /api/contact	Save a contact-form lead + notify admin	Rate-limited
POST /api/b2b	Save a partnership lead + notify partnerships inbox	Rate-limited
POST /api/payment/create-order	Create a Razorpay order (server-priced)	Rate-limited (15/min/IP)
POST /api/payment/verify	Verify signature, provision account + enrollment, log in	HMAC signature is the trust boundary

9. Storage Requirements

9.1 Static assets (public/) — served by the host, not the database

Folder	Size	Contents
public/brochures	≈ 94.0 MB	Downloadable program brochure PDFs
public/video	≈ 70.5 MB	Hero video, Discover-Your-Path video, partners background video
public/program-images	≈ 54.2 MB	Program/course cover images
public/icons	≈ 7.4 MB	Stage icons (Discover/Learn/Mentor/AIRA/Placed)
public/govt-approvals	≈ 2.3 MB	AICTE/APSICHE/ISO/MSME/etc. logos
public/tools-logos	≈ 1.8 MB	Tool/tech logos shown on site
public/partners-network	≈ 1.8 MB	Partner college/company logos
public/alumni-logos	≈ 1.4 MB	Alumni employer logos
public/careers	≈ 0.8 MB	Job-opening description PDFs
Total public/	≈ 238 MB	All static media the site serves

This folder is committed to git and deployed with the app; a Node.js host or its CDN serves it directly. No user uploads exist today, so this figure grows only when the team adds new media.

9.2 Database storage

Neon Serverless PostgreSQL stores only small structured rows (course metadata, leads, users, enrolments) — no media/binary content lives in the database. A free/small Neon tier is more than sufficient at current scale; storage grows slowly with each new user, lead, and enrolment row.

9.3 Local development footprint

Item	Size	Committed to git?
node_modules/	≈ 455 MB	No — regenerated via npm install
.next/ (build cache/output)	Varies (100s of MB)	No — regenerated via npm run build
public/ (assets)	≈ 238 MB	Yes — required at runtime
src/ (source code)	< 1 MB	Yes

9.4 Recommendation

If repository size becomes a concern as more media is added, consider moving large binary assets (videos, brochures) out of git into an object store / CDN (e.g. Vercel Blob, Cloudflare R2, AWS S3) and referencing them by URL instead of bundling them in public/. Not required at the current ~238 MB scale, but worth planning for.

10. Environment Variables Required

Defined in `.env.local` for local development (never committed to git) and must be configured again in the hosting provider's dashboard for production. **Only variable names are listed below — actual secret values are intentionally omitted from this document.**

Variable	Used for	Required?
DATABASE_URL	Neon Postgres connection string	Optional (site falls back to seed data without it)

Variable	Used for	Required?
AUTH_SECRET	Signs session cookies (HMAC-SHA256)	Required for login/dashboard to work
SMTP_HOST / SMTP_PORT	Outgoing mail server	Optional (emails silently skipped if unset)
SMTP_USER / SMTP_PASS	Mail auth credentials	Optional
SMTP_FROM	"From" address (defaults to SMTP_USER)	Optional
ADMIN_EMAIL	Lead-notification inbox	Optional (defaults to SMTP_USER)
PARTNERSHIPS_EMAIL	B2B lead inbox	Optional (defaults to ADMIN_EMAIL)
RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET	Server-side payment order + verify	Required for checkout to work
NEXT_PUBLIC_RAZORPAY_KEY_ID	Client-side checkout widget (same value as KEY_ID)	Required for checkout to work
NEXT_PUBLIC_SITE_URL	Absolute links inside emails	Optional
NEXT_PUBLIC_BASE_PATH	Sub-path hosting (e.g. GitHub Pages project path)	Optional — leave unset on Vercel/Netlify
GEMINI_API_KEY	One-off local script (scripts/generate-images.mjs) only	Not read by the running app

11. Deployment Requirements

`next.config.ts` explicitly documents that this app **cannot** use Next.js static export (output: "export") because it has real server-side routes (login, payment, lead capture) and the `/dashboard` auth gate. It needs a host that runs Node.js.

Requirement	Detail
Runtime	Node.js-capable host (Vercel, Netlify with Next.js runtime, Render, etc.) — not plain static hosting like classic GitHub Pages
Build command	<code>npm install && npm run build</code>
Start command	<code>npm run start</code> (or the host's Next.js adapter)
Database	A reachable Neon Postgres instance + <code>DATABASE_URL</code> set, with <code>scripts/schema.sql</code> applied once via <code>npm run db:migrate</code>
Payments	Live Razorpay keys configured for real transactions (test keys otherwise)
Email	SMTP credentials configured for lead/enrolment emails to send
Config file present	<code>netlify.toml</code> exists in the repo, indicating Netlify is the currently intended host

— End of document —